

Senior Node.js Developer — Take-Home Assignment

Before you start

You may use AI tools. Claude, ChatGPT, Copilot — all fine. We use them too.

What we are assessing is not whether you can produce this code. It is whether you understand what you produced, and whether your judgement is good where the requirements run out.

You will be asked to defend every decision in a follow-up call.

Time budget: 2–3 hours. This is a ceiling, not a target. If you finish in ninety minutes, stop. We would rather see a small, well-reasoned submission than a large one you cannot explain.

The task

Build an endpoint that returns a user's order history.

GET /api/users/:id/orders

A Postgres schema and a seed script are provided (~5,000 users, ~50,000 orders).

Requirements

1. Returns the user's orders, newest first
1. Must stay responsive as the number of orders grows
1. Handle users who have no orders
1. Only the user themselves, or an admin, may view a user's orders

That is the entire specification. It is deliberately incomplete — as real tickets usually are.

What to deliver?

1. Working code

A running service, with whatever tests you think are warranted.

2. DECISIONS.md

This is the part we read first. Answer these four questions:

1. What did the requirements not tell you?

List every assumption you had to make. Mark the one you are least confident about.

2. What did you use AI for, and where did you override it?

Be specific. Which parts were generated, what did the tool get wrong, and what did you change? "Used it for boilerplate" tells us nothing.

3. What breaks first at 100× this data?

Name the specific failure. How would you detect it before a customer reports it?

4. What did you deliberately not build?

Every submission omits something. We want to see that the omissions were chosen rather than overlooked.

How is this graded?

DECISIONS.md carries roughly 60% of the weight. We grade it before opening your code.

What do we look for?

- **Assumptions:** Several identified, with the risky one flagged
- **AI usage:** Concrete account of what the tool got wrong and what you changed
- **Scale:** A specific failure mode and how you would detect it
- **Scope:** Deliberate omissions, with reasoning
- **Code:** Correct, readable, and consistent with what you claimed

A candidate with average code and sharp reasoning will outrank one with polished code they cannot explain. Code review can fix code. Nothing fixes absent judgement.



Submission

Reply to this email with a repository link (or a zip). Include setup instructions — if we cannot run it in five minutes, we will grade it as it arrives.

The follow-up call

Twenty minutes, screen shared. We will pick lines from your submission and ask why. Expect questions like:

- Why this structure rather than the obvious alternative?
- What happens on this line if the input is empty?
- The requirement just changed to include pagination — where does that go?
- If you had to cut this in half, what goes?

Bring your DECISIONS.md. It is the agenda.

Questions about the assignment are welcome — asking a good one is itself a positive signal.